

Faculty Of Engineering and Technology
Department of Computer Science

(Lecture-03)

JavaScript DOM

- The **document object** represents the whole html document.
- When html document is loaded in the browser, it becomes a document object. It is the **root element** that represents the html document. It has properties and methods. By the help of document object, we can add dynamic content to our web page.

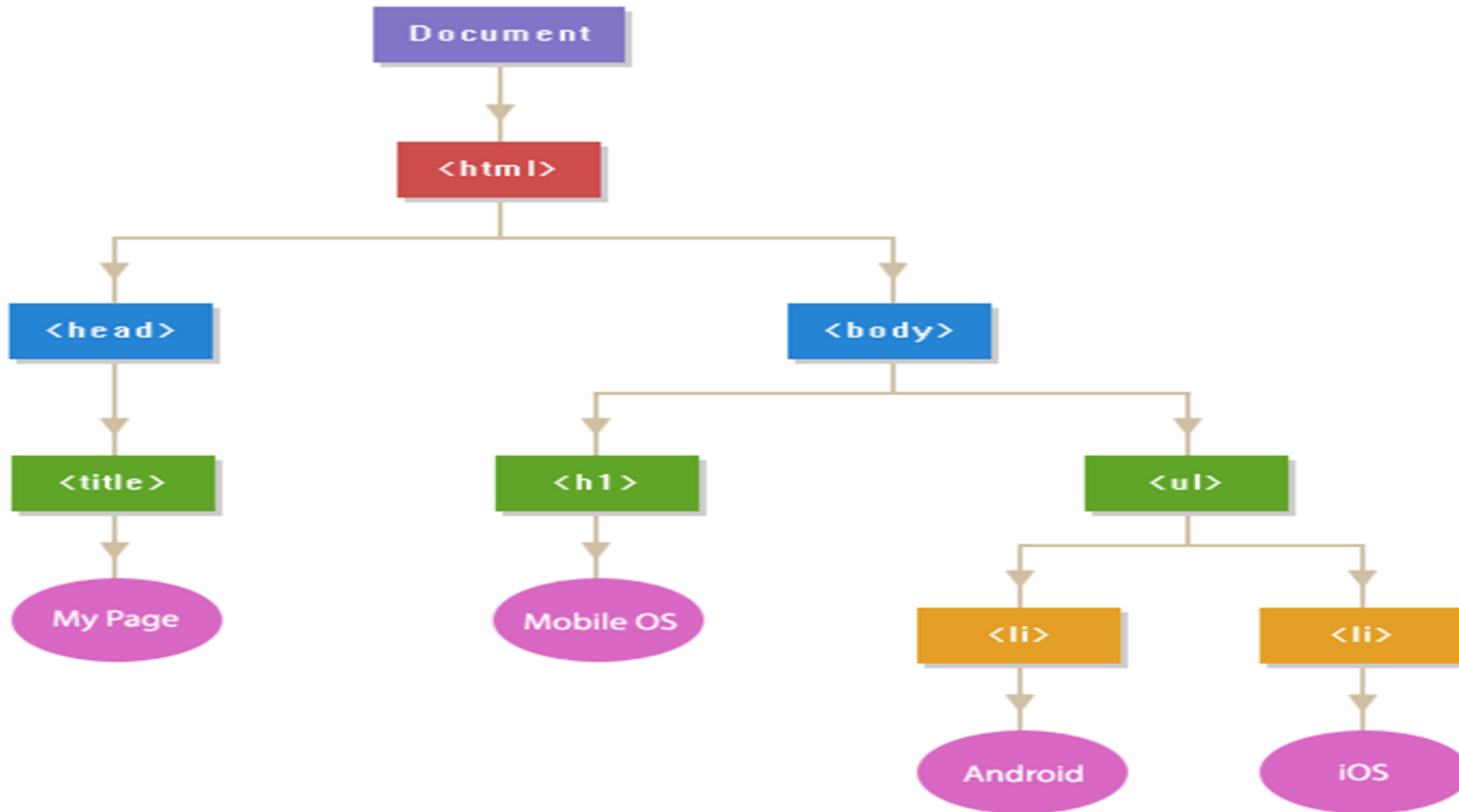
JavaScript DOM

- The Document Object Model, or DOM for short, is a platform and language independent model to represent the HTML or XML documents.
- It defines the logical structure of the documents and the way in which they can be accessed and manipulated by an application program.
- In the DOM, all parts of the document, such as elements, attributes, text, etc. are organized in a hierarchical tree-like structure; similar to a family tree in real life that consists of parents and children *nodes*.

JavaScript DOM

```
<!DOCTYPE html>
<html>
<head>
  <title>My Page</title>
</head>
<body>
  <h1>Mobile OS</h1>
  <ul>
    <li>Android</li>
    <li>iOS</li>
  </ul>
</body>
</html>
```

JavaScript DOM



DOM Nodes and Relationships

- Document node — represents the complete document.
- Element node — represents HTML elements such as `<p>`, `<div>`, and `<button>`.
- Text node — represents text inside an element.
- Attribute — information associated with an element, such as `id`, `class`, `src`, or `href`.
- Parent — a node that contains another node.
- Child — a node contained by another node.
- Sibling — nodes that share the same parent.

The document Object

- document is the main entry point for accessing the DOM.
- It represents the loaded HTML document.
- JavaScript uses document to find elements, create elements, modify content, and work with the page.
- Example: document.title reads or changes the page title.

```
console.log(document.title);  
document.title = "My JavaScript Page";
```

Methods of document object

`getElementById()` — returns the element having the given id value.

`getElementsByName()` — returns all elements having the given name value.

`getElementsByTagName()` — returns all elements having the given tag name.

`getElementsByClassName()` — returns all elements having the given class name.

`querySelector()` — uses a CSS selector and returns the first matching element.

`querySelectorAll()` — uses a CSS selector and returns all matching elements.

getElementById

- Use an element's unique id to select one element.
- If no element has the requested id, the result is null.

```
<p id="abc"></p>
<button onclick="getById()">GetById</button>

<script>
function getById() {
    document.getElementById("abc").innerText = "This is getElementById method";
}
</script>
```

getElementsByName()

Selects elements whose name attribute matches the supplied value.

It can return multiple elements, so iteration is commonly used.

```
<div name="names">First Deve</div>
<div name="names">Second Deve</div>

<script>
function getbyname() {
    var data = document.getElementsByName("names");
    for (var x = 0; x < data.length; x++) {
        alert(data[x].innerText);
    }
}
</script>
```

getElementsByTagName()

Selects elements by their HTML tag name.

For example, `getElementsByTagName('li')` finds all list-item elements.

```
<ol>
  <li>Green</li>
  <li>Blue</li>
  <li>Yellow</li>
  <li>White</li>
</ol>

<script>
function getbytagname() {
  var value = document.getElementsByTagName("li");
  for (var x = 0; x < value.length; x++) {
    alert(value[x].innerText);
  }
}
</script>
```

getElementsByClassName()

Selects all elements that contain the specified class name.

The result can contain multiple elements, so iteration is common.

```
<h5 class="laptops">DEEL</h5>
<h5 class="laptops">HP</h5>
<h5 class="laptops">Apple</h5>

<script>
function getbyclassname() {
    var inputs = document.getElementsByClassName("laptops");
    for (var x = 0; x < inputs.length; x++) {
        alert(inputs[x].innerText);
    }
}
</script>
```

querySelector()

querySelector() uses a CSS selector to find an element.

It returns the first element that matches the selector.

It can select by tag, id, class, attribute, or more complex CSS selector.

```
const first = document.querySelector(".laptop");  
if (first) {  
    first.innerText = "First laptop";  
}
```

querySelector() — Common CSS Selectors

- `document.querySelector('#title')` — first element with `id=title`.
- `document.querySelector('.item')` — first element with `class=item`.
- `document.querySelector('p')` — first paragraph.
- `document.querySelector('ul li')` — first `li` inside a `ul`.

querySelectorAll()

querySelectorAll() uses CSS selectors and returns all matching elements.

It is useful when several elements must be changed or inspected.

The returned NodeList can be iterated with forEach().

```
const items = document.querySelectorAll(".item");

items.forEach(function(item) {
  item.innerText = "Updated item";
});
```

querySelector() vs querySelectorAll()

- `querySelector()` → first matching element.
- `querySelectorAll()` → all matching elements.
- Both accept CSS selector syntax.
- Example: `querySelector('.box')` changes only the first `.box`; `querySelectorAll('.box')` can process every `.box`.

Changing Text — innerText

innerText gets or sets the visible text of an element.

HTML markup assigned through innerText is treated as ordinary text rather than HTML.

It is useful for messages, labels, validation feedback, and visible text updates.

```
function chagetext() {  
    document.getElementById("loc").innerText =  
        "Please do not clear this text";  
}  
  
<input type="submit"  
    value="change the text in the paragraph"  
    onclick="chagetext()">  
  
<p id="loc">This is the text in the paragraph</p>
```

Changing HTML — innerHTML

innerHTML gets or sets the HTML markup contained inside an element.

It is useful for inserting dynamic HTML such as forms, links, lists, and formatted content.

Do not insert untrusted user input directly with innerHTML because it can create an XSS vulnerability.

```
function showCommentBox() {  
    var data = "Your Comment Here : " +  
        "<textarea rows='10' cols='100'></textarea><br>" +  
        "<input type='submit' value='Post Comment'>";  
  
    document.getElementById("mylocation").innerHTML = data;  
}  
  
<input type="submit" value="Click here for comment"  
    onclick="showCommentBox()">  
  
<div id="mylocation"></div>
```

innerText vs innerHTML

- innerText = text content visible to the user.
- innerHTML = HTML markup plus its text content.
- innerText assigned 'Hello' displays the characters as text.
- innerHTML assigned 'Hello' renders bold text.
- For plain user-provided text, textContent is often preferable.

textContent

textContent gets or sets the text content of a node.

Unlike innerHTML, it does not parse the assigned value as HTML.

It is a useful choice when the goal is simply to set or read text.

```
const message = document.querySelector("#message");  
message.textContent = "<strong>Hello</strong>";  
// The tags appear as text; they are not rendered as HTML.
```

Changing HTML Elements

- `element.innerHTML = new html content` — changes the inner HTML of an element.
- `element.attribute = new value` — changes the attribute value of an HTML element.
- `element.style.property = new style` — changes the style of an HTML element.

These techniques allow JavaScript to update the page dynamically.

Attributes

HTML attributes provide additional information about an element.

Common attributes include id, class, href, src, alt, title, disabled, and data-* attributes.

JavaScript can read, change, add, and remove attributes.

```
const link = document.querySelector("a");  
  
console.log(link.getAttribute("href"));  
link.setAttribute("href", "https://example.com");  
link.removeAttribute("title");
```

Direct Attribute Properties

Many common attributes can also be accessed through element properties.

Examples: `img.src`, `input.value`, `input.disabled`, `link.href`.

Use `getAttribute()/setAttribute()` when generic attribute access is needed.

```
const image = document.querySelector("img");
image.alt = "University building";

const input = document.querySelector("input");
input.value = "Khalil";
input.disabled = false;
```

Classes — className

className gets or sets the complete class attribute.

Assigning className replaces the existing class list.

Use classList when you want to work with individual classes.

```
const box = document.querySelector("#box");  
box.className = "box active";
```

Classes — classList

classList provides methods for individual CSS classes.

add() — adds a class.

remove() — removes a class.

toggle() — adds the class if absent or removes it if present.

contains() — checks whether a class exists.

replace() — replaces one class with another.

```
const box = document.querySelector("#box");

box.classList.add("active");
box.classList.remove("hidden");
box.classList.toggle("selected");

console.log(box.classList.contains("active"));
```

Changing Styles

The style property changes inline CSS styles.

JavaScript CSS property names use camelCase: backgroundColor, fontSize, marginTop.

For many style changes, using CSS classes with classList is cleaner and easier to maintain.

```
const title = document.querySelector("h1");  
  
title.style.color = "blue";  
title.style.fontSize = "32px";  
title.style.backgroundColor = "lightgray";
```

Creating Elements

`document.createElement(tagName)` creates a new DOM element.

The new element exists in memory until it is inserted into the document.

Set its text, attributes, classes, or children before insertion.

```
const li = document.createElement("li");  
li.textContent = "Windows";  
li.classList.add("os-item");  
  
console.log(li);
```

Inserting Elements

`appendChild()` adds a node as the last child.

`append()` can add nodes and text.

`prepend()` adds content at the beginning.

`before()` and `after()` insert content around an existing element.

```
const list = document.querySelector("#osList");
const li = document.createElement("li");

li.textContent = "Windows";
list.appendChild(li);
```

DOM Manipulation — Complete Workflow

1. Select an existing element.
2. Read its content or attributes if necessary.
3. Change text, HTML, attributes, styles, or classes.
4. Create new elements when additional content is needed.
5. Insert the new elements into the DOM.
6. Remove elements when they are no longer needed.

JavaScript Validation

JavaScript can inspect form values before submission.

DOM methods are used to select inputs and display validation messages.

Client-side validation improves user experience but must not be treated as a security boundary.

The server must validate untrusted input again.

```
const username = document.querySelector("#username");
const message = document.querySelector("#message");

if (username.value.trim() === "") {
    message.textContent = "Username is required.";
} else {
    message.textContent = "Valid input.";
}
```

Thank You...!